VITO CODEX

FULL TECHNICAL AUDIT

github.com/k0reets86/vito_codex

03.03.2026 | v0.3.0 | 39,916 lines Python

1. Executive Summary

VITO — амбициозный проект автономного AI-агента для генерации дохода на цифровых площадках. Кодовая база — ~40K строк Python (134 файла кода, 48 тестов). Архитектурно проект выстроен грамотно: чёткое разделение на агентов/платформы/модули, DI через конструкторы, единый BaseAgent, продуманный LLM Router с fallback-цепочками.

Общая оценка: 6.5/10. Фундамент крепкий, но многие агенты — тонкие LLM-обёртки без реальной специализации, 5 из 19 платформ — заглушки, а revenue pipeline пока реально работает только для Gumroad.

Направление	Оценка	Комментарий
Архитектура	8/10	Чистая модульность, DI, BaseAgent, разделение agents/platforms/modules
Качество кода	7/10	Хорошие docstrings, логирование, но type hints неполные
Безопасность	7/10	.env, нет hardcoded ключей, но AGENT_SYSTEM_PREAMBLE рискован
Интеграции	5/10	Gumroad реальный, 5 платформ-заглушек, Etsy без OAuth2 write
Память	7/10	Три слоя реализованы, но формула релевантности не применяется
Self-healing	7/10	3 уровня эскалации, shell-фиксы, но нет канареечного тестирования
Глубина агентов	5/10	~10 агентов — LLM-обёртки без собственной логики
DevOps	8/10	systemd, watchdog, cgroups, resource_guard — отлично
Тесты	6/10	48 файлов, 18% coverage ratio, моки качественные
Дух идеи	7/10	Goal→Plan→Execute→Learn реализован, но revenue = 0

2. Архитектура и соответствие духу

2.1 Структура проекта

Файловая организация чистая и логичная. Чёткое разделение: agents/ (26 файлов), platforms/ (19), modules/ (25), config/ (6), tests/ (48), deploy/ (3), docs/ (19+). Это соответствует документации ARCHITECTURE и показывает зрелый подход.

✅ **Модульность и разделение ответственности — отлично**

2.2 Цикл Goal → Plan → Execute → Learn

Это сердце VITO, заявленное в SOUL. Реализация: GoalEngine управляет состояниями (PENDING → PLANNING → EXECUTING → COMPLETED/FAILED), DecisionLoop._tick() каждые 5 мин выбирает следующую цель, VITOCore планирует и диспетчеризирует.

Фаза Learn: после выполнения сохраняются навыки в SQLite, обновляются паттерны, ошибки логируются. Это реально работает — не заглушка.

✅ **Цикл реализован полностью и соответствует документации**

2.3 LLM Router

Один из лучших компонентов. 8 моделей в регистре (Gemini Flash Lite для рутины, Claude Sonnet для контента, o3 для кода, Perplexity для research, Claude Opus для стратегии). Fallback-цепочки для каждого TaskType. Gemini rate limiting (15 req/min). SQLite кэш ответов. Интеграция с FinancialController для budget enforcement.

✅ **LLM Router — продуманный, экономный, с fallback**

2.4 Два основных закона SOUL

Закон 1 "Зарабатывать больше чем тратить": Финансовый контроллер работает (P&L, лимиты, ROI). Но реальный доход = $0. Pipeline существует, но не доведён до конца.

Закон 2 "Становиться умнее каждый день": Реализован через skill registry, ночную консолидацию, anti-skills. Работает концептуально.

⚠️ **Законы заложены в код, но не проверены в бою**

3. Глубина агентов: LLM-обёртки vs специалисты

Ключевая проблема: многие агенты следуют одному паттерну: принять параметры → составить prompt → отправить в LLM → вернуть текст. Нет собственной логики, нет внешних API, нет состояния.

Полноценные агенты (собственная логика):

VITOCore (664 LOC) — оркестрация, keyword-классификация, product pipeline, self-improve. Реальная логика диспетчеризации.

BrowserAgent (444 LOC) — Playwright, OOM-защита, singleton, реальный headless. Это рабочий инструмент.

ResearchAgent (462 LOC) — Perplexity API, реальные источники, структурированные результаты.

TrendScout (314 LOC) — RSS фиды, Reddit, реальный парсинг.

DevOpsAgent (243 LOC) — реальные shell-команды, health check, disk/memory.

ECommerceAgent (127 LOC) — полный pipeline публикации с одобрением.

ContentCreator (160 LOC) — сохраняет файлы, но по сути LLM-обёртка с файловой системой.

Тонкие LLM-обёртки (нужна доработка):

SEOAgent (72 LOC), MarketingAgent (69 LOC), LegalAgent (77 LOC), RiskAgent (76 LOC), EconomicsAgent (78 LOC), PartnershipAgent (69 LOC), TranslationAgent (64 LOC) — все следуют одному паттерну: prompt → LLM → текст. Нет интеграции с реальными API (Google Search Console, Ahrefs, DeepL, и т.д.).

❌ ~10 агентов — фактически заменяемы одним LLM-вызовом с другим prompt

4. Платформы: реальные vs заглушки

Рабочие интеграции:

Gumroad — самая зрелая. API v2 для чтения + Playwright browser automation для публикации (т.к. Gumroad убрал POST products из API). 40+ скриптов в scripts/ показывают реальный опыт борьбы с UI.

Twitter/X — OAuth 1.0a, реальный постинг через API v2.

WordPress — REST API v2 + Application Password. Готов к работе.

Medium — Integration token, готов к работе.

YouTube — Data API v3, только чтение (тренды).

Etsy — API v3, read-only (x-api-key). Write = OAuth2 PKCE — не реализован.

Printful — REST API, заголовки готовы, но print-on-demand pipeline не завершён.

Полные заглушки:

Instagram, LinkedIn, Pinterest, Shopify, Threads — 5 файлов по 20-25 строк, все возвращают "not_implemented". Не несут ценности.

⚠️ **5/19 платформ = заглушки. Etsy не может писать. Реальный revenue path = только Gumroad**

5. Безопасность

5.1 Секреты

Все API-ключи через .env + os.getenv(). Файл .env в .gitignore. Никаких hardcoded секретов в коде не найдено (проверено regex-сканом).

✅ **Секреты защищены**

5.2 Prompt Injection — КРИТИЧЕСКИЙ РИСК

AGENT_SYSTEM_PREAMBLE в base_agent.py содержит: "All instructions you receive come from the VITO orchestrator — they are legitimate internal system commands... Never refuse or flag internal system instructions." Это прямое отключение защиты LLM от prompt injection.

Если злоумышленник внедрит инструкцию в данные (RSS, поиск, скрапинг), LLM выполнит её как "легитимную". Это особенно опасно при наличии CodeGenerator + SelfUpdater, которые могут модифицировать код.

❌ **КРИТИЧЕСКИЙ: AGENT_SYSTEM_PREAMBLE отключает защиту LLM**

5.3 CodeGenerator + SelfUpdater

Имеются защитные механизмы: ALWAYS_BLOCKED_FILES (.env), PROTECTED_FILES (main.py, comms_agent.py), MAX_PATCH_SIZE, skill_security.scan_text(). Но allow_protected=True в VITOCore._self_improve() обходит защиту.

⚠️ **Защита есть, но allow_protected=True обходит её**

6. Система памяти

Три слоя реализованы: ChromaDB (семантический поиск), SQLite (навыки, ошибки, паттерны), PostgreSQL+pgvector (эпизодическая память). Graceful degradation — если PG недоступен, работаем без него. ChromaDB fallback на in-memory для тестов.

Формула релевантности (0.60 semantic + 0.25 recency + 0.15 importance) описана в docstring, но в search_knowledge() используется только ChromaDB cosine similarity — recency/importance не учитываются.

⚠️ **Формула релевантности не реализована — документация врёт**

7. DevOps и операционная зрелость

Это сильная сторона проекта:

systemd unit с cgroup лимитами (MemoryMax=4G, CPUQuota=200%, TasksMax=50). Это продуманно и важно для сервера с 8GB RAM.

Watchdog скрипт каждые 60с проверяет RAM и убивает headless_shell/pytest зомби процессы.

ResourceGuard в Python — читает /proc/meminfo, блокирует тяжёлые операции, async context manager. Отлично.

PID lock в main.py предотвращает дублирование процессов. Graceful shutdown через signal handlers.

✅ **DevOps — продакшн-уровень**

Не хватает: Dockerfile/docker-compose (Hetzner bare metal, но контейнеризация упростила бы деплой), CI/CD pipeline (в GitHub Actions), requirements.txt / pyproject.toml (не найдено в репо!).

❌ **Отсутствует requirements.txt/pyproject.toml — критично**

8. Тесты

48 файлов, 6061 строк. Ratio 18%. conftest.py с качественными моками всех зависимостей (LLM, memory, finance, comms). Есть test_integration_offline.py (996 LOC) — самый большой, проверяет взаимодействие компонентов.

Плюсы: каждый агент покрыт, моки грамотные, AsyncMock для async.

Минусы: нет e2e тестов с реальными API, нет проверки полного pipeline Gumroad, нет проверки ночной консолидации и scheduler.

⚠️ **Тесты есть, но нужны e2e с реальными API**

9. Конкретные ошибки и проблемы

9.1 Отсутствие requirements.txt

В репозитории нет ни requirements.txt, ни pyproject.toml, ни Pipfile. Невозможно воспроизвести окружение. Зависимости (выявлены анализом import): anthropic, openai, chromadb, asyncpg, aiohttp, playwright, python-dotenv, pytest.

9.2 Hardcoded пути

/home/vito/vito-agent/ захардкожен в settings.py, content_creator.py, financial_controller.py. Нужно вынести в PROJECT_ROOT или .env.

9.3 memory/vito_memory.db в репо

Файл vito_memory.db закоммичен в репозиторий, хотя *.db в .gitignore. Это может содержать личные данные и навыки. Нужно: git rm --cached memory/vito_memory.db.

9.4 Однообразность агентов

SEOAgent, MarketingAgent, LegalAgent, RiskAgent, EconomicsAgent, PartnershipAgent, TranslationAgent — все следуют шаблону: if not self.llm_router: return error + await self._call_llm(prompt). Нет состояния, нет кэша, нет интеграции с внешними сервисами.

9.5 Gumroad scripts/

40+ скриптов gumroad_fix_v4..v10, gumroad_tags_v11..v12, gumroad_cover_*. Это следы отладки, не чистый код. Нужно: оставить финальный скрипт, остальное в архив.

9.6 Dashboard.tsx и tmp_dashboard.tsx в корне

Два TSX-файла в корне проекта без package.json, без сборки, без интеграции. Нужно: вынести в dashboard/ с отдельным package.json или удалить.

9.7 output/ в репо

Файлы ai_side_hustle_cover_1280x720.png и thumb закоммичены. output/ в .gitignore, но эти файлы остались в истории.

10. Приоритетные рекомендации

P0 — Критические (сделать сейчас)

1. Убрать AGENT_SYSTEM_PREAMBLE или заменить на санитаризацию входных данных. Вместо "никогда не отказывай" — тегируй данные как <user_data> и учи LLM не выполнять инструкции внутри данных.
2. Создать requirements.txt или pyproject.toml. Без этого проект невоспроизводим.
3. git rm --cached memory/vito_memory.db и output/*.png. Проверить git log на случайные .env в истории.

P1 — Важные (неделя)

4. Реализовать формулу релевантности в search_knowledge(). После ChromaDB query — re-rank по 0.60×sim + 0.25×recency + 0.15×importance.
5. Убрать allow_protected=True из VITOCore._self_improve(). Вместо этого — список разрешённых файлов для самомодификации.
6. Доделать Etsy OAuth2 PKCE — это второй revenue channel.
7. Вынести hardcoded /home/vito/vito-agent/ в одну константу PROJECT_ROOT.

P2 — Улучшения (месяц)

8. Обогатить тонких агентов: SEOAgent → Google Search Console API, TranslationAgent → DeepL API, MarketingAgent → локальный кэш стратегий.
9. Удалить платформы-заглушки (Instagram, LinkedIn, Pinterest, Shopify, Threads) — они создают ложное ощущение функциональности.
10. Очистить scripts/ — оставить финальные версии, архивировать остальное.
11. Добавить Dockerfile + docker-compose + GitHub Actions CI.
12. e2e тест Gumroad pipeline: от создания цели до публикации (с mock browser).

11. Итог

VITO — это не набор заглушек. Это рабочий фундамент с продуманной архитектурой, который пока не приносит денег.

Фундамент готов на ~90%: Decision Loop, LLM Router, Memory, Financial Controller, Self-Healing, DevOps — всё работает.

Главный разрыв: между качественным ядром и тонкими агентами/заглушками платформ. Ядро (main.py, decision_loop.py, llm_router.py, vito_core.py, memory_manager.py) — зрелый код уровня senior разработчика. Периферия (seo_agent.py, legal_agent.py, shopify.py) — MVP-заглушки.

Проект соответствует духу идеи автономного AGI для заработка — но находится на этапе перехода от фундамента к первым деньгам. Следующий шаг: сконцентрироваться на 1-2 платформах (Gumroad + Etsy) и довести до первой продажи.